PRACTICAL SESSION 7

Task 1: Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

```
  UW PICO 5.09                          File: memory.c

#include <stdio.h>
#include <stdlib.h>

int global_var = 10;
static int static_var = 20;

int main() {
    int stack_var = 30;
    int *heap_var = malloc(sizeof(int));
    *heap_var = 40;

    printf("Code address:   %p\n", (void *)main);
    printf("Global address: %p\n", (void *)&global_var);
    printf("Static address: %p\n", (void *)&static_var);
    printf("Heap address:   %p\n", (void *)heap_var);
    printf("Stack address:  %p\n", (void *)&stack_var);

    free(heap_var);
    return 0;
}
```

```
(base) saiabburi@Charishma-MacBook-Air ~ % mkdir Practical7
mkdir: Practical7: File exists
(base) saiabburi@Charishma-MacBook-Air ~ % cd Practical7
(base) saiabburi@Charishma-MacBook-Air Practical7 % nano memory_layout.c
(base) saiabburi@Charishma-MacBook-Air Practical7 % clang -Wall -Wextra memory_layout.c -o memory_layout
(base) saiabburi@Charishma-MacBook-Air Practical7 % ./memory_layout
----- Memory Layout -----
Address of code/function : 0x102153cbc
Global variable address  : 0x102158000
Static variable address  : 0x102158004
Heap variable address    : 0x600000a18020
Stack variable address   : 0x16dcaf568

Variable values:
Global = 100
Static = 200
Heap   = 400
Stack  = 300
```

Task 2: Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
(base) saiabburi@Charishma-MacBook-Air Practical7 % nano memory_process.c
(base) saiabburi@Charishma-MacBook-Air Practical7 % clang -Wall -Wextra memory_process.c -o memory_process
(base) saiabburi@Charishma-MacBook-Air Practical7 % ./memory_process
Process ID (PID): 1263
Global address  : 0x100dd8000
Stack address   : 0x16f02f558
Heap address    : 0x14a604430
Program is running. Press Control+C to stop.
^C
(base) saiabburi@Charishma-MacBook-Air Practical7 %
```

```
(base) saiabburi@Charishma-MacBook-Air ~ % vmmap 1263
Process:         memory_process [1263]
Path:            /Users/USER/*/memory_process
Load Address:    0x100dd0000
Identifier:      memory_process
Version:         ???
Code Type:       ARM64
Platform:        macOS
Parent Process:  zsh [1231]

Date/Time:       2026-10-13 10:49:37.007 +0530
Launch Time:     2026-10-13 10:49:01.718 +0530
OS Version:      macOS 13.4.1 (22F82)
Report Version:  7
Analysis Tool:   /usr/bin/vmmap

Physical footprint:         913K
Physical footprint (peak):  913K
Idle exit:                  untracked
----

Virtual Memory Map of process 1263 (memory_process)
Output report format:  2.4  -- 64-bit process
VM page size:  16384 bytes
```

```
(base) saiabburi@Charishma-MacBook-Air ~ % ls -l ~/Practical7/memory_map_report.txt
-rw-r--r--  1 saiabburi  staff  0 Oct 13 10:56 /Users/saiabburi/Practical7/memory_map_report.txt
(base) saiabburi@Charishma-MacBook-Air ~ %
```

```
  UW PICO 5.09                                                File: memory_process.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_data = 10;

int main(void) {
    int local_data = 20;
    int *heap_data = malloc(100 * sizeof(int));

    if (heap_data == NULL) {
        perror("malloc");
        return 1;
    }

    heap_data[0] = 30;

    printf("Process ID (PID): %d\n", getpid());
    printf("Global address  : %p\n", (void *)&global_data);
    printf("Stack address   : %p\n", (void *)&local_data);
    printf("Heap address    : %p\n", (void *)heap_data);

    printf("Program is running. Press Control+C to stop.\n");
    fflush(stdout);

    while (1) {
        sleep(10);
    }

    free(heap_data);
    return 0;
}
```